

Lab #1

Introduction to PIC18F assembly language programming

ECE 3301 Introduction to Microcontrollers Laboratory

Sarah Boyd

6/15/25

Dr. Tamer Omar



**Cal Poly
Pomona**

College of Engineering

Execution videos and program files:

Prelab i:

- Output/ analysis video:
<https://www.youtube.com/watch?v=dVEU2m89DO8>

Prelab ii:

- Output/analysis video:
<https://youtu.be/kC8HMqk1IbM>

Post lab

- Output/analysis video:
<https://www.youtube.com/watch?v=BX9IUDbHS18>

Objective:

The purpose of this lab is to assemble and debug a PIC18F assembly language program using the Microchip's MPLAB x 4.20 assembler.

Background analysis:

Before diving into writing assembly code, it is important to understand the fundamentals of writing assembly code. Although this lab provides memory addresses and diagrams, students need to understand concepts like WREG and SFR registers and carry flags. Writing correct (and efficient) code cannot be done without understanding core concepts. This section of the lab report aims to cover the basic knowledge utilized in writing these programs.

To use assembly effectively we must first understand the role of registers and how memory addresses work in a microcontroller. The Central Processing Unit (CPU) consists of three main components: registers, the Arithmetic Logic Unit (ALU), and the control unit. There are 4 basic, fundamental CPU registers- Instruction Register (IR) , Program counter (PC) Accumulator (ACC) and Memory address register (MAR).

The IR (Instruction Register) is used to store instructions after being fetched to be executed. Instructions are originally stored in program memory (PM) (memory used to store instructions). There are two types of registers- data register and address register- and because the IR stores data, it is considered a data address. The contents of the IR are always decoded by the CPU as an instruction. After reading (fetching) an instruction from memory, the CPU stores it in the IR. The CPU then translates the instruction and performs the operation required. Note that the word size of the CPU normally determines the size of the instruction register.

The Program Counter (PC) stores the address of the next ALU instruction to be executed. Since it holds an address, it is considered an address register. The address of the first instruction to be executed is loaded by activating the CPU's reset input. The CPU executes the program sequentially, but instructions like JUMP can change the contents of the PC from its normal sequential address value. The size of the Program Counter is determined by the size of the address bus.

The Accumulator (ACC) stores the result after most arithmetic or logic operations. The accumulator, normally an 8-bit register, is referred to as WREG (working register) and usually

contains one of the operands before an arithmetic or logic operation. CPUs can be classified as either accumulator-based or general-purpose-register-based. In accumulator-based microcontrollers such as the PIC18F, all ALU operations are performed using the accumulator as one of the data sources. In general-purpose-register-based microcontrollers, such as the TI MSP430, any general-purpose register can be used as an accumulator.

The Memory Address Register (MAR) contains the address of the data to be accessed. The CPU uses this address as a direct pointer to memory, and the content at that memory location is the actual data being transferred. For the PIC18F, data memory consists of general-purpose registers (file registers) and selected special function registers (SFRs).

The PIC18F4321 includes both general-purpose registers and SFRs in its memory structure. General-purpose registers store temporary data and carry out data transfers between various registers, which speeds up program execution because the microcontroller does not need to read from external memory. Some common functions performed by GPRs include moving data to or from memory and incrementing or decrementing registers by one. Selected Special Function Registers (SFRs) are dedicated registers mapped into Bank 15 of data memory and are used for control and configuration tasks.

PRELAB:

prelab i) Assemble and execute the tutorial program written in PIC18F assembly language using the MPLAB (appendix F of the textbook). Note that the GUI of MPLAB has changed and the screenshots in the book are only for guidance .

First, the textbook appendix F was used to download MPLAB x 4.20 and understand how to create programs correctly and also debug. This appendix was referenced a lot throughout the lab. Prelab i code was given in this appendix to demonstrate and provide practice for the process. Additionally, there was a powerpoint included on canvas titled PIC18F Instruction Sets Examples that provided conceptual examples as well as further formatting examples for the flow charts.

INCLUDE	<P18F4321.INC>	
SUM	EQU	0x50
	ORG	0x100
	MOVLW	0x02
	ADDLW	0x05
	MOVWF	SUM
HERE	BRA	HERE
END		

IMG 1. *The provided code to compile*

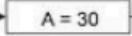
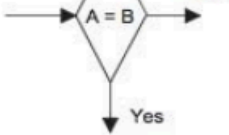
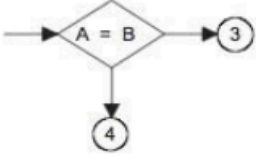
```

1      Include <P18F4321.INC>
2      SUM      EQU      0x50
3
4      ORG      0x100
5
6      MOVLW    0x02
7      ADDLW    0x05
8      MOVWF    SUM
9
10     HERE     BRA      HERE
11     END

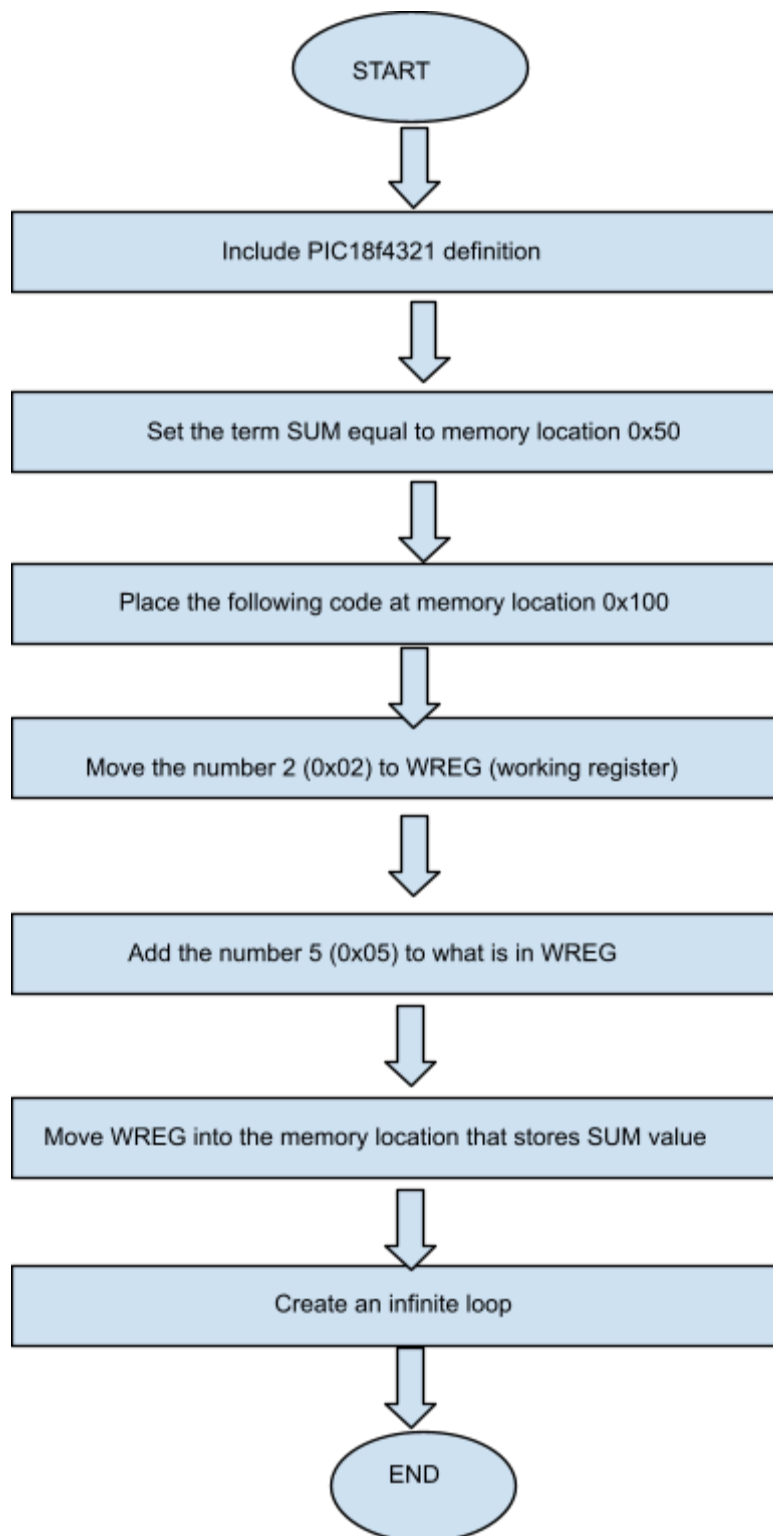
```

IMG 2. *Code*

TABLE 3.3 Flowchart symbols

Symbol	Function	Example
 Rectangle	Operation to be carried out	 Arrow indicates direction of program flow
 Diamond	Logical decision	 Exit diamond from right if $A \neq B$ and from bottom if $A = B$
 Oval	Terminal point (typically start and end of program)	 Start
 Circle	Connection from one point in a flowchart to another	
 Parallelogram	I/O Operation	 Input D

IMG 3. Flowchart symbol key, from the textbook *Microcontroller Theory and Applications with the PIC18F* by M.Rafiquzzaman.

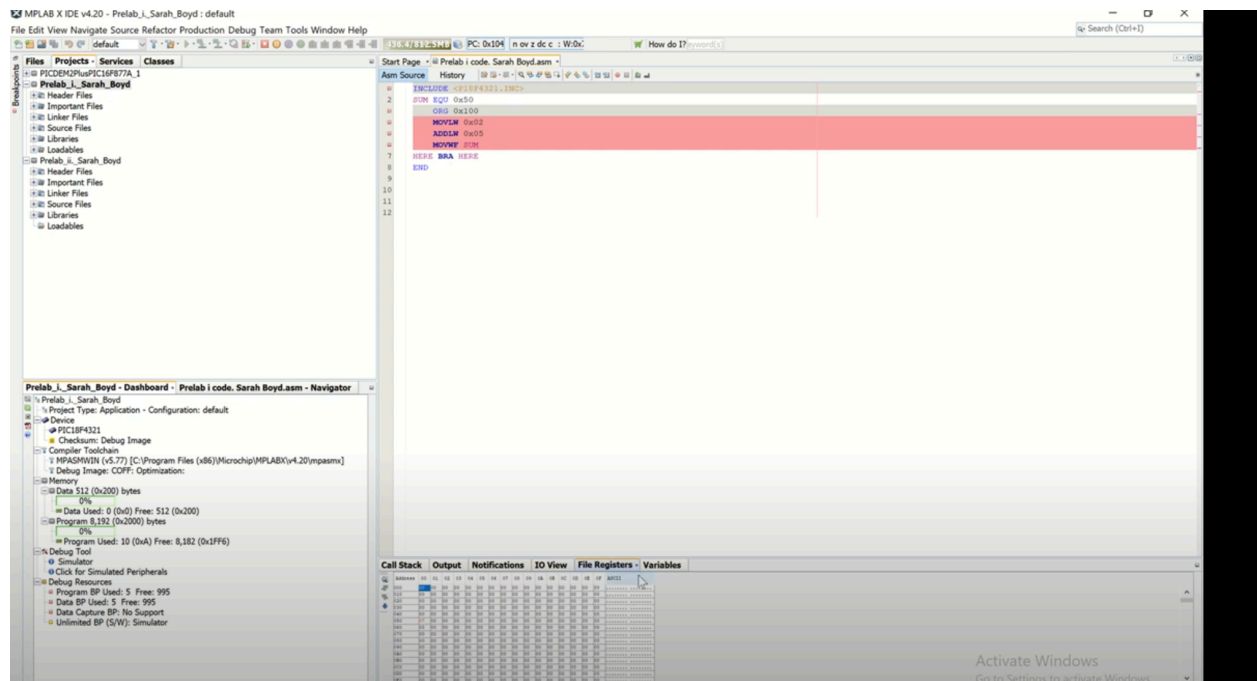


IMG 4. Flow chart for pre lab i.

Here is my reasoning and understanding of the code, line by line.

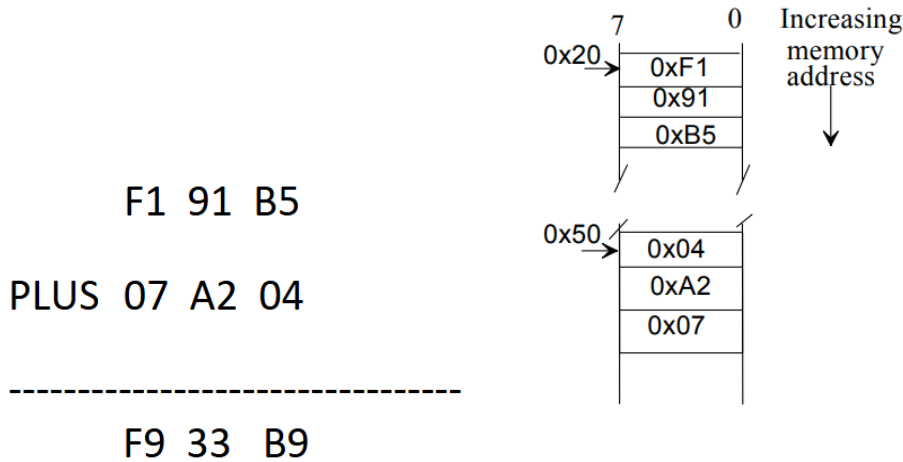
Line	Instruction	Explanation
1	INCLUDE <P18F4321.INC>	Includes definitions for PIC18F4321 specifically- what register names, memory architecture configuration, SFRS, flags, etc
2	SUM EQ 0x50	This line is like saying “the phrase SUM equals/ refers to memory location 0x50”. So when the sum is written, the value should be stored into memory location 0x50. SUM-Pre-chosen symbolic name chosen to represent memory location 0x50 EQ-Directive (not instruction) setting sum equal to the memory location 0x50- The memory address 0x50 is pre-chosen by the textbook.
3	ORG 0x100	“Origin” — this tells the assembler to place the following code starting at program memory address 0x100.
4	MOVLW 0x02	"Move Literal to WREG" — loads the number 2 into the WREG register.
5	ADDLW 0x05	Adds the number 5 to whatever is already in WREG (which is 2), so now WREG holds 7.
6	MOVWF SUM	"Move WREG to File Register" — stores the result in WREG (7) into the memory location labeled SUM (which is 0x50).
7	HERE BRA HERE	BRA = Branch Always. This creates an infinite loop so the program stays running and doesn't go into unknown code. It keeps repeating this line forever.
8	END	Tell the assembler this is the end of your code. Nothing after this will be compiled.

Table 1. Instructions and what they accomplish for the given prelab i code.



IMG 5. The successfully built program, correctly storing (7) in 0x50. More information about the logistics concerning the code execution, registers and access for files can be found on page 2 in the execution and program files section.

prelab ii) It is desired to add two 24- bit data items in memory as shown in the figure below. Store the result pointed to by 0x50. The operand with the sample data is given by the two diagrams below. The memory addresses are predetermined, like in prelab i.



IMG 6. Addition of the two 24-bit data items

IMG 7. Logistics of memory addresses.

Operand 1 (MSB to LSB)= B5 91 F1

	Initial memory location	Byte
F1	0x20	Lowest byte
91	0x21	Medium byte
B5	0x22	Highest byte

Table 2. Operand 1 memory location and byte

Operand 2 (MSB to LSB)= 07 A2 04

	Initial memory location	Byte
0x04	0x50	Lowest byte
0xA2	0x51	Medium byte
0x07	0x52	Highest byte

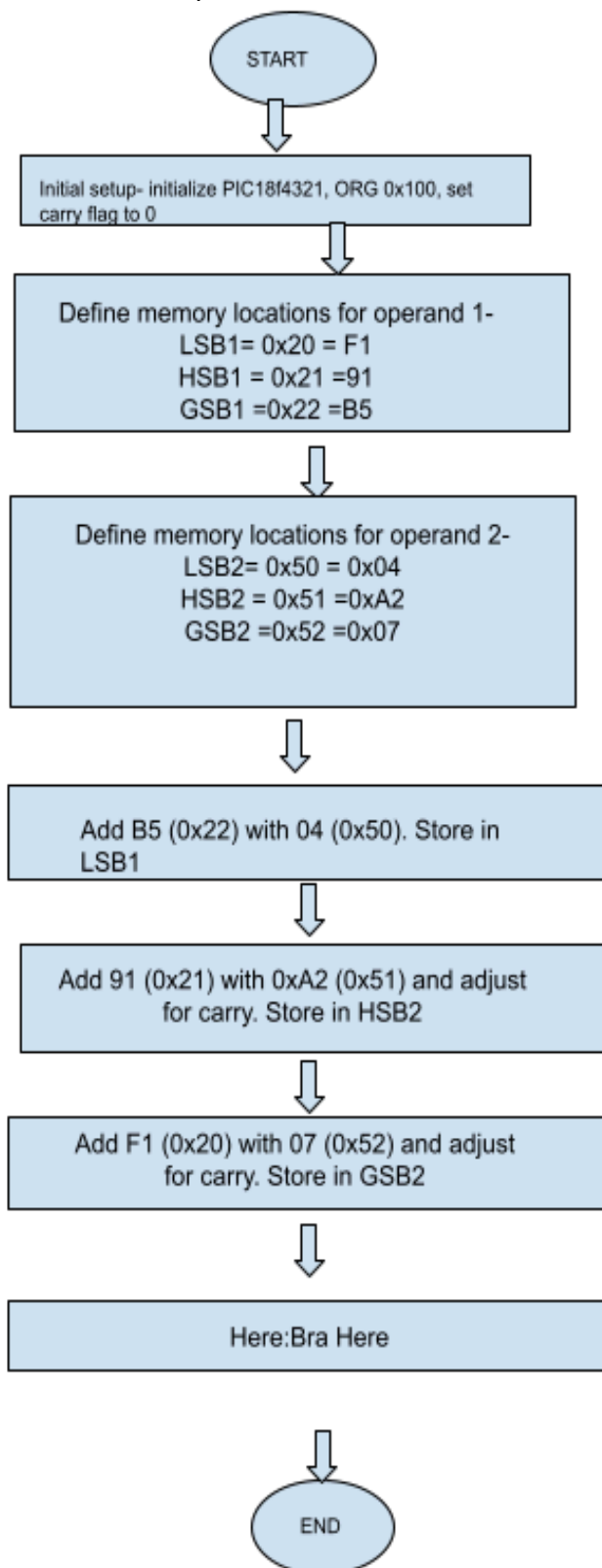
Table 3. Operand 2 memory location and byte

RESULT (MSB to LSB)= F9 33 B9

	Final memory location	Byte
B9	0x50	Lowest byte
33	0x51	Medium byte
F9	0x52	Highest byte

Table 4. Expected result of the code in prelab ii. We expect memory address 0x50 to hold the lowest byte result and memory address 0x52 to hold the highest byte result.

a) Flowchart the problem



- 1) Convert flowchart to PIC18F assembly program at address 0x100. Do not use loop and indirect addressing mode.

Since this program performs multi-byte addition, there can be carrying of the 10s such as in typical multi digit addition. This is why we use ADDWFC (add WREG with file and carry) for more significant bytes instead of just ADDWF (add WREG), which is only suitable for least significant bytes that don't necessitate carrying. The carry flag in the STATUS register (SFR, a special function register)—is automatically set to 1 if there is a carry from an addition or a borrow from a subtraction. If no carry occurs, the flag remains 0. This flag is crucial when performing multi-byte arithmetic, as it helps determine whether an adjustment is needed when adding the next higher byte. Carry handling is managed manually and that the flag can be used throughout the program (hence BTFSC STATUS, 0) as well as conditional incrementing (INCFSZ) rather than using ADDWFC.

The carry flag in the STATUS register—a special function register (SFR)—is automatically set to 1 if there is a carry from an addition or a borrow from a subtraction. If no carry occurs, the flag remains 0. This flag is crucial when performing multi-byte arithmetic, as it helps determine whether an adjustment is needed when adding the next higher byte.

Additionally, when writing and understanding this program it is important to know when to use MOVLW versus MOVF or MOVWF. MOVLW (Move Literal to WREG) loads a literal (constant) value directly into the WREG register. This is useful when initializing values or preparing data for an operation. MOVWF (Move WREG to File Register) takes the contents of WREG and stores it into a specified file register (memory address). On the other hand, MOVF (Move File Register to WREG or File Register) moves the contents of a memory address into WREG or keeps it within the file register depending on the destination bit. In this code, MOVF is used to load values from memory into WREG prior to arithmetic operations.

Note that in this lab, lines 17 to 37 (which load operands into memory using MOVLW and MOVWF) are commented out and not executed. This simplification is possible because the

values are already stored at the specified memory addresses, so there is no need to initialize them manually.

```

16
17     ; Initialize operand 1 (not needed in this case)
18
19     ; MOVLW 0xF1 ;Load WREG with F1
20     ; MOVWF LSB1 ;Store this into LSB address for operand 1 (0x20)
21
22     ; MOVLW 0x91 ;Load WREG with 91
23     ; MOVWF HSB1 ;Store this into HSB address for operand 1 (0x21)
24
25     ; MOVLW 0xB5 ;Load WREG with B5
26     ; MOVWF MSB1 ;Store this into MSB address for operand 1 (0x22)
27
28
29     ; Initialize operand 2 (not needed in this case)
30     ; MOVLW 0x04 ;Load WREG with 4
31     ; MOVWF LSB2 ;Store into address for LSB for operand 2 (0x50)
32
33     ; MOVLW 0xA2 ;Load WREG with A2
34     ; MOVWF HSB2 ;Store this into the address for HSB operand 2 (0x51)
35
36     ; MOVLW 0x07 ;Load WREG with 07
37     ; MOVWF MSB2 ;Store into address for MSB operand 2 (0x52)
38
39

```

IMG 9. Unnecessary initialization of variables

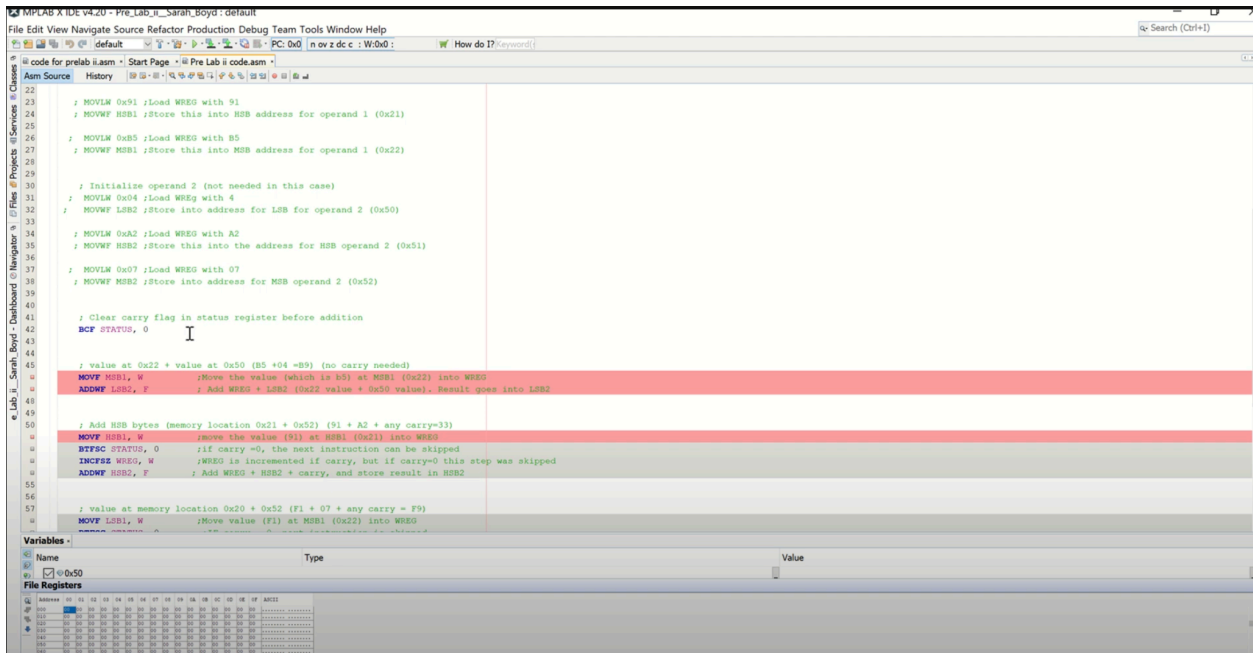
Here is the code with line by line explanations.

line	Instruction	Explanation
1	INCLUDE <P18F4321.INC>	Includes definitions for PIC18F4321 specifically- what register names, memory architecture configuration, SFRS, flags, etc
2	PSECT PL2	Declaring program section named pl2
3	ORG 0x100	Tells microcontroller to start placing the program's machine instructions starting at memory address 0x100. The "starting point" of the program
4	LSB1 EQU 0x20	We start by defining memory location for operand 1. Defining memory location for operand 1. This line defines the term LSB 1 to the memory location of the most significant bit (0x22, holding the value B5)his line defines the term LSB1 to the memory location of the least significant bit (0x20, holding the value F1).
5	HSB1 EQU 0x21	Defining memory location for operand 1. This line defines the term HSB1 to the memory location of the byte with the middle significance (0x21) holding

		the value 91).
6	MSB1 EQU 0x22	Defining memory location for operand 2. This line defines the term MSB 1 to the memory location of the most significant bit (0x22, holding the value B5)
y	LSB2 EQU 0x50	Defining memory location for operand 2. This line defines the term LSB2 to the memory location of the most significant bit (0x50, holding the value 4).
8	HSB2 EQU 0x51	Defining memory location for operand 2. This line defines the term HSB2 to the memory location of the byte with the middle significance (0x51) holding the value A2).
9	MSB2 EQU 0x52	Defining memory location for operand 2. This line defines the term MSB2 to the memory location of the most significant bit (0x52, holding the value 7)
10	BCF STATUS, 0	Clears carry flag (in status register) before we do addition in later lines.
11	MOVF MSB1, W	Our next goal is to add value at 0x22 to the value at 0x50 (B5 +04 =B9) (no carry needed). First, we write this line of code to move the value (which is b5) at MSB1 (0x22) into WREG.
12	ADDWF LSB2, F	Add WREG + LSB2 (0x22 value + 0x50 value). The result goes into LSB2
13	MOVF HSB1, W	Our next goal is to add the data at memory location 0x21 (91) to the value at memory location 0x51 (A2) plus any carry, to get 33. First, We move the value (91) at HSB1 (0x21) into WREG.
14	BTFSC STATUS, 0	If carry flag =0, the next instruction will be skipped.
15	INCF SZ WREG, W	If carry flag is 1, the working register is incremented
16	ADDWF HSB2, F	We add the value in the working register to the value at HSB2 plus carry, and then store the result in HSB2.
17	MOVF LSB1, W	Our next goal is to add the value at memory location 0x20 (F1) to the value at the memory location 0x52 (07) to any carry, and get F9. To do this, we first move value (F1) at MSB1 (0x22)

		into WREG
18	BTFSC STATUS, 0	.If carry flag =0, the instruction on the line after this can be skipped.
19	INCFSZ WREG, W	Assuming carry =1, WREG is increment.
20	ADDWF MSB2, F	We add the value in the working register (F1) and the value at MSB2 (memory location 0x50, with value 07) plus carry and store in location MSB2 (0x51)
	HERE: BRA HERE	BRA = Branch Always. This creates an infinite loop so the program stays running and doesn't go into unknown code. It keeps repeating this line forever.
12	END	Tell the assembler this is the end of your code. Nothing after this will be compiled.

Table 5. An in-depth analysis of the code from prelab ii.



IMG 10. Working code from prelab ii. Note that register analysis, output and debugging is seen in the video located in execution and program files section.

POSTLAB:

4. Equipment, Software and Components required:

Microchip's MPLAB assembler/ Debugger

5. Description (corresponding topics covered in the textbook)

The concepts associated with this lab provide familiarization with basic assembly language programming in MPLAB and with the MPLAB assembler/debugger commands

Prerequisites: Basics of assembly language programming, PIC18F registers , PIC18F Memory Organization, PIC18F Data movement instructions, Addition instructions, MPLAB assembler/Debugger tutorial.

6. Procedure: Assemble the PIC18F assembly language program using the MPLAB. Then use the debugging tool to verify that the program works properly.

7. Deliverables:

PostLab: Write a PIC18F assembly program for Prelab using loop and indirect addressing mode

Previously we used direct addressing mode, where the exact memory address was specified in each instruction. This approach is straightforward and easy to understand because you can clearly see which memory locations are being accessed. However, it becomes tedious and inflexible when working with large data sets or repetitive operations, as each address must be manually specified. For example, it saves a lot of time if you have to iterate through 100 address locations because you'd have to list all 100 addresses if you used direct addressing mode. In indirect addressing, you store the address you are targeting in a special pointer register and the instructions use that register to access the real memory location. Although this is harder to follow, it is easier to not type so many addresses, easier to work with loops, arrays, blocks of data and adds flexibility to an otherwise rigid program. Essentially, addresses are stored in the FSR register, and INDF is used to access the value at that address. You can combine loops and indirect addressing to repeat the same operation over a sequence of

memory addresses and iterate through memory instead of saying each memory address location.

Recall that WREG is the working register, like a temporary holding area (think of it like your hand when you're adding two numbers).MOVLW means "Move Literal to WREG" — you're loading a number into WREG.MOVWF means "Move WREG to File register" — you're storing that number into memory.ADDWF means "Add WREG to File Register" — it adds two numbers and stores the result.ADDWFC is the same as ADDWF but also includes the carry from the last addition (used when adding multi-byte numbers).BCF STATUS, 0 clears the carry bit — this is very important before starting a new multi-byte addition.

Goal: Add the two 24-bit numbers (using the same memory address format) and store the result in the least significant byte, medium significance byte, and most significant byte locations reserved for the second operand. The process is the same as before, but the difference is that this time we use indirect addressing with pointer registers (FSR0 and FSR1) and looping, rather than explicitly specifying each memory address. We are expecting the same resultant output as in prelab ii.

Line	Instruction	Goal
1	INCLUDE <P18F4321.INC>	Includes definitions for PIC18F4321 specifically- what register names, memory architecture configuration, SFRS, flags, etc
2	ORG 0x00	Tells microcontroller to start placing the program's machine instructions starting at memory address 0x100. The "starting point" of the program
3	GOTO START	This prompts the program to jump immediately to start, and we have to do this because the real program code is later.
4	ORG 0x100	Sets the place where our main program code will be stored and run.
5	SRC_LSB EQU 0x20	SRC_LSB is our name that stands for source least significant bit and is the location where the smallest byte of the source is stored in memory (location 0x20)
6	DEST_LSB EQU 0x50	Stands for destination of the least significant bit, and is the memory location where the smallest byte of the destination number or result is stored. So this line gives the second number's lowest byte a name and memory location (address 0x50), also where result goes.
7	COUNT EQU 0x60	Count is a memory address where we'll store how many bytes we still

		need to add. Since our numbers are 3 bytes long, we will later set this to 3.
8	START:	Marks the spot where the main program starts running (so the chip knows where to begin). Where we go after GOTO START command
9	LFSR 0, SRC_LSB + 2	Sets pointer 0 (FSR0) to point at the first number's highest byte (address 0x22).
10	LFSR 1, DEST_LSB	Sets pointer 1 (FSR1) to point at where the second number's lowest byte is (address 0x50)
11	BCF STATUS, C	Clears the carry flag (makes sure there's no leftover extra bit from before we add).
12	MOVLW 0x03	Loads the number 3 into the working register (because we have 3 bytes to add).
13	MOVWF COUNT	Saves that 3 into our counter spot so we know how many bytes to add.
14	ADD_LOOP:	Labels this spot as the start of the loop that will add each byte one by one.
15	MOVF POSTDEC0, W	POSTDEC0 = de-increment FSR0 by 1, moving the pointer down to the next lower byte. MOVF means move file register to WREG. SO we read the current byte from the source number then move the pointer down to the next byte.
16	ADDWFC POSTINC1, F	ADDWFC= add wreg and carry to file register, postinc1 means use FSR1 (pointing to result byte). We add WREG + current value at FSR1 and carry bit, then increment FSR1 by 1 (moving pointer up to next higher byte).
17	DECFSZ COUNT, F	DECFSZ = decrement file register and skip if 0, we are decreasing count by 1. If after its decreasing its 0, we skip to the next instruction. And basically the point of doing this is to keep track of how many bytes are left to add.
18	GOTO ADD_LOOP	If count is not zero, we jump back to add loop command to add the next byte. This loops three times total, once per byte
19	END_LOOP:	Label for a spot where the program will stop doing anything (infinite wait).
20	BRA END_LOOP	BRA (branch always), creates an infinite loop and keeping the chip busy so it doesn't run random memory after finishing
21	END	Tells the assembler this is the end of the program.

Report: Submit a final Lab report using blackboard, the report should include all of the following except in case of simulation deliverable (d) is not required.

- a. Submit the source code (.asm or .c files) according to the lab requirements.
- b. Submit a snapshot for the pic memory views or watches for the variables that validate the program output.
- c. Submit a word report for any required flow charts.
- d. Submit a picture for your Hardware assembled circuit.

8. Concluding remarks:

Complete each lab and make sure it is checked by the instructor.